

Sushmit Partakke

23070521156

SQL JOIN Practice Questions

Assume you have the following two tables:

Employees

emp_id	name	department_id	salary
1	Alice	101	5000
2	Bob	102	6000
3	Charlie	NULL	5500
4	Dave	101	7000

Departments

department_id	department_name
101	HR
102	IT
103	Marketing

Basic JOINS

1. Write a query to get all employees along with their department names.

```
SELECT e.emp_id, e.name AS employee_name, d.department_name, e.salary
FROM Employees e
LEFT JOIN Departments d ON e.department_id = d.department_id;
```

emp_id	employee_name	department_name	salary
1	Alice	HR	5000
2	Bob	IT	6000
3	Charlie	NULL	5500
4	Dave	HR	7000

2. List the names of employees who **do not belong** to any department.

```
SELECT *
FROM Employees
WHERE department_id IS NULL;
```

```
employee_name
-----
Charlie
```

3. List all departments and the names of employees in them (if any).

```
SELECT d.department_id, d.department_name, e.name AS employee_name
```

```
FROM Departments d
```

```
LEFT JOIN Employees e ON d.department_id = e.department_id;
```

department_id	department_name	employee_name
101	HR	Alice
101	HR	Dave
102	IT	Bob
103	Marketing	NULL

INNER, LEFT, RIGHT, FULL JOINS

4. Use an **INNER JOIN** to find employees who are assigned to a department.

```
BEGIN
  FOR rec IN (
    SELECT e.emp_id, e.name AS employee_name, d.department_name
    FROM Employees e
    INNER JOIN Departments d ON e.department_id = d.department_id
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('Emp ID: ' || rec.emp_id ||
      ', Name: ' || rec.employee_name ||
      ', Dept: ' || rec.department_name);
  END LOOP;
```

END;

Emp ID	Name	Dept
1	Alice	HR
2	Bob	IT
4	Dave	HR

5. Use a **LEFT JOIN** to list all employees and their department names (even if department is NULL).

BEGIN

```
FOR rec IN (  
  SELECT e.emp_id, e.name AS employee_name, d.department_name  
  FROM Employees e  
  LEFT JOIN Departments d ON e.department_id = d.department_id  
) LOOP  
  DBMS_OUTPUT.PUT_LINE('Emp ID: ' || rec.emp_id ||  
    ', Name: ' || rec.employee_name ||  
    ', Dept: ' || NVL(rec.department_name, 'NULL'));
```

END LOOP;

END;

6. Use a **RIGHT JOIN** to list all departments and employees (if any).

BEGIN

```
FOR rec IN (  
  SELECT d.department_id, d.department_name, e.name AS employee_name  
  FROM Departments d  
  LEFT JOIN Employees e ON d.department_id = e.department_id  
) LOOP  
  DBMS_OUTPUT.PUT_LINE('Dept ID: ' || rec.department_id ||  
    ', Dept Name: ' || rec.department_name ||  
    ', Employee: ' || NVL(rec.employee_name, 'NULL'));
```

END LOOP;

END;

Dept ID: 101, Dept Name: HR, Employee: Alice

Dept ID: 101, Dept Name: HR, Employee: Dave

Dept ID: 102, Dept Name: IT, Employee: Bob

Dept ID: 103, Dept Name: Marketing, Employee: NULL

7. Use a **FULL OUTER JOIN** to show all employees and all departments, matching where possible.

```
BEGIN
  FOR rec IN (
    SELECT e.emp_id, e.name AS employee_name, d.department_name
    FROM Employees e
    FULL OUTER JOIN Departments d ON e.department_id = d.department_id
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('Emp ID: ' || NVL(TO_CHAR(rec.emp_id), 'NULL') ||
                          ', Name: ' || NVL(rec.employee_name, 'NULL') ||
                          ', Dept: ' || NVL(rec.department_name, 'NULL'));
  END LOOP;
END;
```

Emp ID: 1, Name: Alice, Dept: HR

Emp ID: 2, Name: Bob, Dept: IT

Emp ID: 3, Name: Charlie, Dept: NULL

Emp ID: 4, Name: Dave, Dept: HR

Emp ID: NULL, Name: NULL, Dept: Marketing

JOIN with Conditions

8. Write a query to list employees who earn more than any other employee in their department.

```
BEGIN
  FOR rec IN (
    SELECT e.emp_id, e.name AS employee_name, d.department_name, e.salary
    FROM Employees e
    JOIN Departments d ON e.department_id = d.department_id
    WHERE e.salary = (
      SELECT MAX(e2.salary)
      FROM Employees e2
      WHERE e2.department_id = e.department_id
    )
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('Emp ID: ' || rec.emp_id ||
      ', Name: ' || rec.employee_name ||
```

```

        ', Dept: ' || rec.department_name ||
        ', Salary: ' || rec.salary);
    END LOOP;
END;

```

Emp ID: 2, Name: Bob, Dept: IT, Salary: 6000
 Emp ID: 4, Name: Dave, Dept: HR, Salary: 7000

9. List all departments where **no employees** are assigned.

```

BEGIN
    FOR rec IN (
        SELECT d.department_id, d.department_name
        FROM Departments d
        LEFT JOIN Employees e ON d.department_id = e.department_id
        WHERE e.emp_id IS NULL
    ) LOOP
        DBMS_OUTPUT.PUT_LINE('Dept ID: ' || rec.department_id ||
            ', Dept Name: ' || rec.department_name);
    END LOOP;
END;

```

Dept ID: 103, Dept Name: Marketing

10. For each department, show the number of employees and the average salary.

```

BEGIN
    FOR rec IN (
        SELECT d.department_id, d.department_name,
            COUNT(e.emp_id) AS num_employees,
            ROUND(AVG(e.salary)) AS avg_salary
        FROM Departments d
        LEFT JOIN Employees e ON d.department_id = e.department_id
        GROUP BY d.department_id, d.department_name
    ) LOOP
        DBMS_OUTPUT.PUT_LINE('Dept ID: ' || rec.department_id ||

```

```

        ', Dept Name: ' || rec.department_name ||

        ', Employees: ' || rec.num_employees ||

        ', Avg Salary: ' || NVL(rec.avg_salary, 0));

    END LOOP;

END;
```

Dept ID: 101, Dept Name: HR, Employees: 2, Avg Salary: 6000

Dept ID: 102, Dept Name: IT, Employees: 1, Avg Salary: 6000

Dept ID: 103, Dept Name: Marketing, Employees: 0, Avg Salary: 0

SQL Trigger Practice Questions

Assume you have a table:

Orders

order_id	customer_name	amount	status
1	John Doe	200	pending
2	Alice Smith	350	shipped

And a log table:

Order_Log

log_id	order_id	operation	changed_at

Trigger Practice

1. Create a trigger that inserts a row into `Order_Log` **after an order is inserted** into `Orders`.

```
CREATE TRIGGER log_order_insert
AFTER INSERT ON Orders
FOR EACH ROW
BEGIN
    INSERT INTO Order_Log (order_id, operation, changed_at)
    VALUES (NEW.order_id, 'INSERT', NOW());
END;

INSERT INTO Orders (order_id, customer_name, amount, status)
VALUES (3, 'Bob Johnson', 150, 'processing');
| log_id | order_id | operation | changed_at      |
|-----|-----|-----|-----|
| 1      | 3       | INSERT   | 2023-11-15 10:30:00 |
```

2. Create a trigger that logs an update to the `Orders` table whenever the `status` changes.

```
CREATE TRIGGER log_status_change
AFTER UPDATE ON Orders
FOR EACH ROW
BEGIN
    IF OLD.status != NEW.status THEN
        INSERT INTO Order_Log (order_id, operation, changed_at)
        VALUES (NEW.order_id, CONCAT('STATUS_CHANGE from ', OLD.status, ' to ',
NEW.status), NOW());
    END IF;
END;

UPDATE Orders SET status = 'delivered' WHERE order_id = 2;
| log_id | order_id | operation                                | changed_at      |
|-----|-----|-----|-----|
| 2      | 2       | STATUS_CHANGE from shipped to delivered | 2023-11-15 10:32:00 |
```

3. Write a trigger that prevents inserting an order with a negative `amount`.

```

CREATE TRIGGER prevent_negative_amount
BEFORE INSERT ON Orders
FOR EACH ROW
BEGIN
    IF NEW.amount < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Order amount cannot be negative';
    END IF;
END;

INSERT INTO Orders (order_id, customer_name, amount, status)
VALUES (4, 'Invalid Order', -100, 'pending');

```

Error Code: 1644. Order amount cannot be negative

4. Write a trigger that automatically sets the `status` to "pending" if no status is provided during insert.

```

CREATE TRIGGER set_default_status
BEFORE INSERT ON Orders
FOR EACH ROW
BEGIN
    IF NEW.status IS NULL THEN
        SET NEW.status = 'pending';
    END IF;
END;

INSERT INTO Orders (order_id, customer_name, amount)
VALUES (5, 'Default Status', 250);

```

order_id	customer_name	amount	status
5	Default Status	250	pending

5. Create a trigger that updates another table `Customer_Balance` whenever an order is placed.

```

CREATE TABLE Customer_Balance (
    customer_name VARCHAR(100) PRIMARY KEY,

```



```
    balance DECIMAL(10,2) DEFAULT 0
);
```

```
INSERT INTO Customer_Balance (customer_name, balance)
VALUES ('John Doe', 0), ('Alice Smith', 0), ('Bob Johnson', 0);
```

```
CREATE TRIGGER update_customer_balance
AFTER INSERT ON Orders
FOR EACH ROW
BEGIN
    INSERT INTO Customer_Balance (customer_name, balance)
    VALUES (NEW.customer_name, NEW.amount)
    ON DUPLICATE KEY UPDATE balance = balance + NEW.amount;
END;
```

```
INSERT INTO Orders (order_id, customer_name, amount, status)
VALUES (6, 'New Customer', 400, 'processing');
```

```
| customer_name | balance |
|-----|-----|
| John Doe     | 200     |
| Alice Smith  | 350     |
| Bob Johnson  | 150     |
| Default Status| 250     |
| New Customer | 400     |
```